

IT 315

IRIS FLOWER CLASSIFICATION

GROUP MEMBERS :

MAJD ALFAYEZ 441203356

RAFA ALDBEEB 432205471

SHROG AL-HARBI 432206874

RUBA ALHAMDAN 431201749

DR. AFAF SALMI



INTRODUCTION

Flower classification stands out as one of machine learning's most compelling real-world applications, enabling algorithms to distinguish between plant species based purely on numerical measurements .

This project focuses on classifying Iris flowers into three species : Setosa, Versicolor, and Virginica , using four physical features: sepal length, sepal width, petal length, and petal width.

The foundation of this work is the classic Iris dataset , a benchmark in the machine learning community containing 150 evenly distributed samples across all three species. The implementation leverages Python with key libraries: pandas and numpy for data processing , matplotlib for visualization, and scikit-learn for model training and evaluation .

Beyond the technical pipeline, the project embraces Object-Oriented Programming principles by structuring the codebase into dedicated classes, each handling a distinct responsibility. This design promotes clarity, maintainability, and scalability. The end goal is a model that reliably predicts a flower's species from its physical measurements alone.

OBJECTIVES

- Classify Iris flowers into three species: Setosa, Versicolor, and Virginica
- Analyze and explore the dataset using data visualization techniques
- Preprocess and split the dataset into training and testing sets
- Train a machine learning model such as Logistic Regression.
- Evaluate the performance of the model using accuracy metrics
- Apply Object-Oriented Programming (OOP) principles in organizing the code
- Design a simple and structured application using classes
- Develop a graphical user interface (GUI) using Tkinter for user interaction

DATASET DESCRIPTION

The Iris dataset is among the most widely recognized benchmarks in the machine learning field. It consists of 150 flower samples split equally across three species : Setosa, Versicolor, and Virginica , with 50 samples representing each class.

Every sample is described by four numerical features, all measured in centimeters:

Sepal Length

Sepal Width

Petal Length

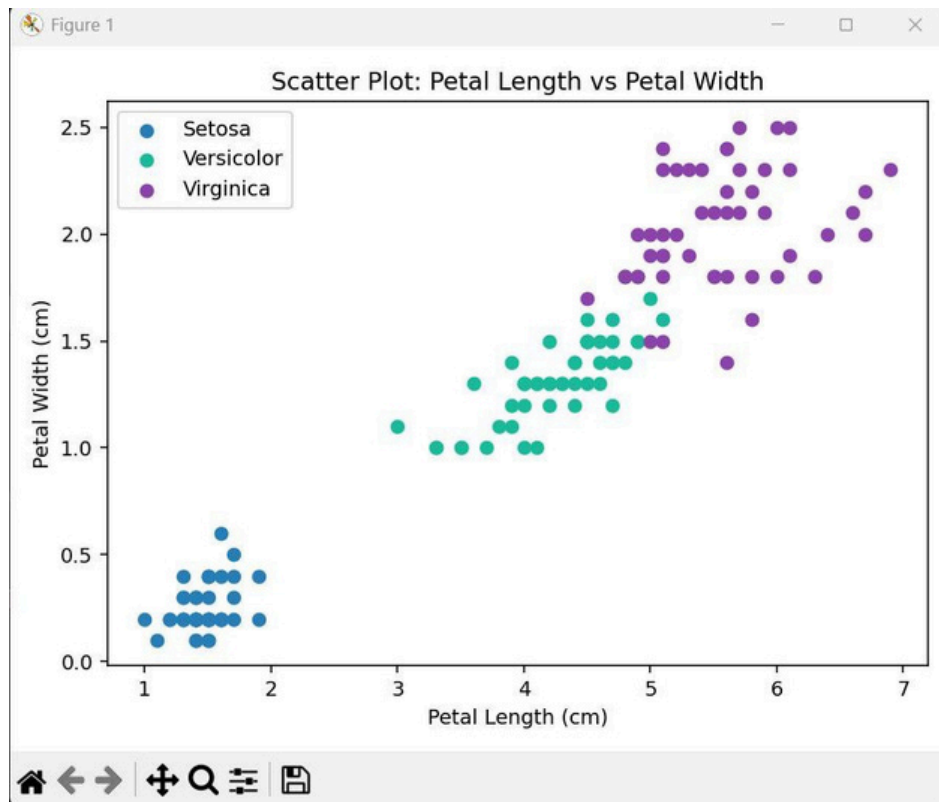
Petal Width

The dataset is notably clean, containing no missing values, which makes it immediately suitable for analysis without requiring extensive preprocessing. It was loaded directly into the project using Python's `sklearn.datasets` library.

METHODOLOGY

- Loading the Iris dataset
- Data preprocessing (handling data and checking for missing values)
- Data visualization (scatter plots, histograms, boxplots)
- Data splitting into training and testing sets (80% training, 20% testing)
- Model selection: Logistic Regression
- Model training using training data
- Model evaluation using accuracy score
- Building a GUI for prediction using Tkinter

DATA ANALYSIS



Scatter Plot (Petal Length vs Petal Width)

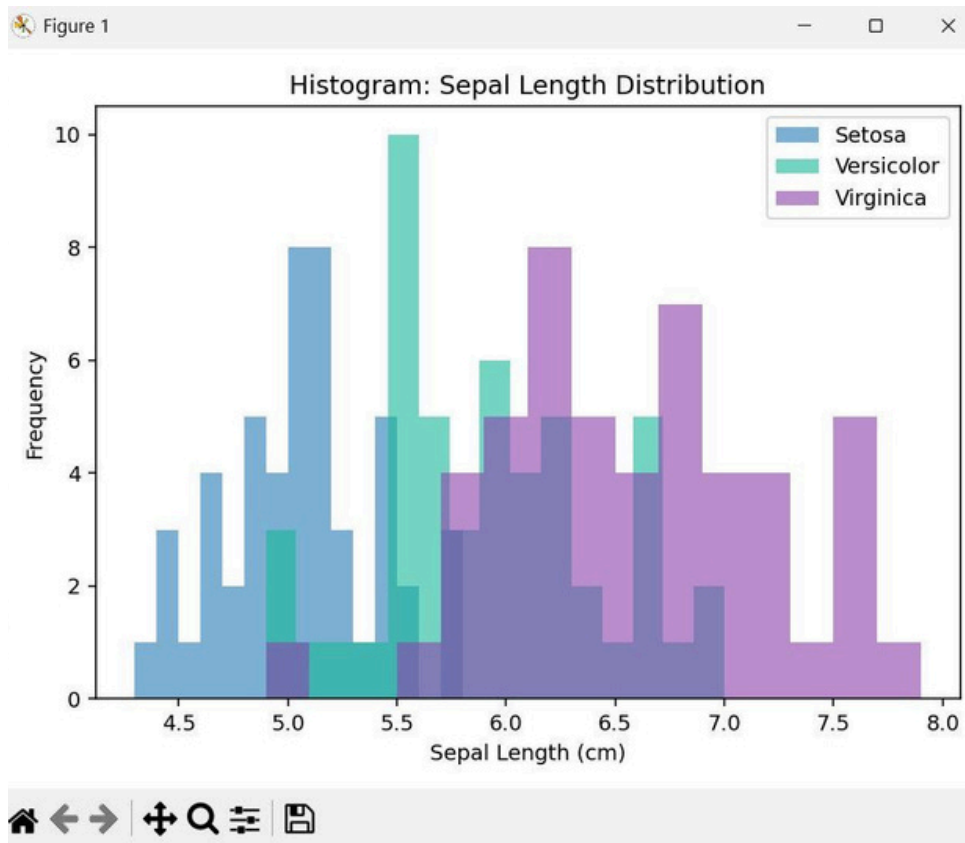
The plot shows a strong positive correlation between petal length and petal width.

It is clear that:

- Setosa is completely separated with very small values
- Versicolor lies in the middle
- Virginica has the largest values

This indicates that these two features are highly effective for distinguishing between species.

DATA ANALYSIS



Histogram (Sepal Length Distribution)

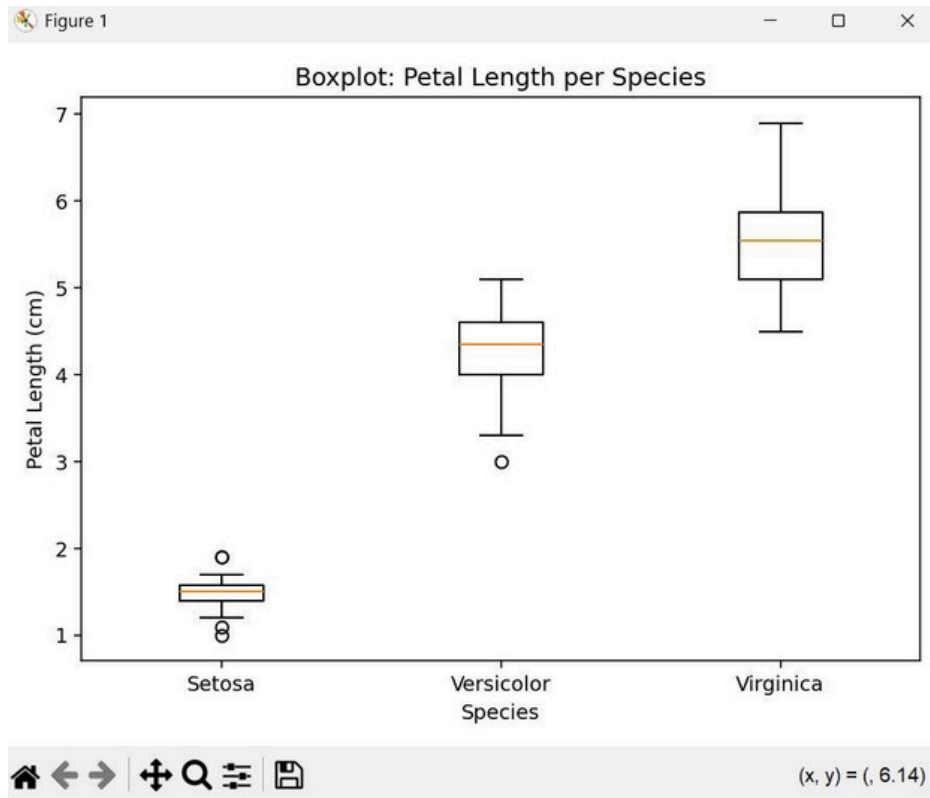
The histogram shows the distribution of sepal length for each species.

Observations:

- There is some overlap between Versicolor and Virginica
- Setosa tends to have smaller values

This suggests that sepal length is less effective for classification compared to petal features.

DATA ANALYSIS



Boxplot (Petal Length per Species)

The boxplot clearly shows differences between species.

Observations:

- Each species has a distinct range
- Setosa has the smallest values
- Virginica has the largest values

This confirms that petal length is one of the most important features for classification .

MODEL IMPLEMENTATION

```
def load_data(self): 1 usage
    iris = load_iris()
    self.df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
    self.df['species'] = iris.target
    print("Data loaded and converted to DataFrame successfully.")
    return self.df
```

The Iris dataset is loaded and converted into a DataFrame. A new column called “species” is added to represent the target labels.

```
def split_data(self): 1 usage
    print("\n--- Model Training Process ---")
    X = self.df.drop('species', axis=1)
    y = self.df['species']

    self.X_train, self.X_test, self.y_train, self.y_test = train_test_split(
        *arrays: X, y, test_size=0.2, random_state=42
    )
```

The dataset is divided into input features (X) and target labels (y), then split into 80% training data and 20% testing data.

```
self.model = LogisticRegression(max_iter=200)
self.model.fit(self.X_train, self.y_train)
```

A Logistic Regression model is created and trained using the training data to learn how to classify the Iris species.

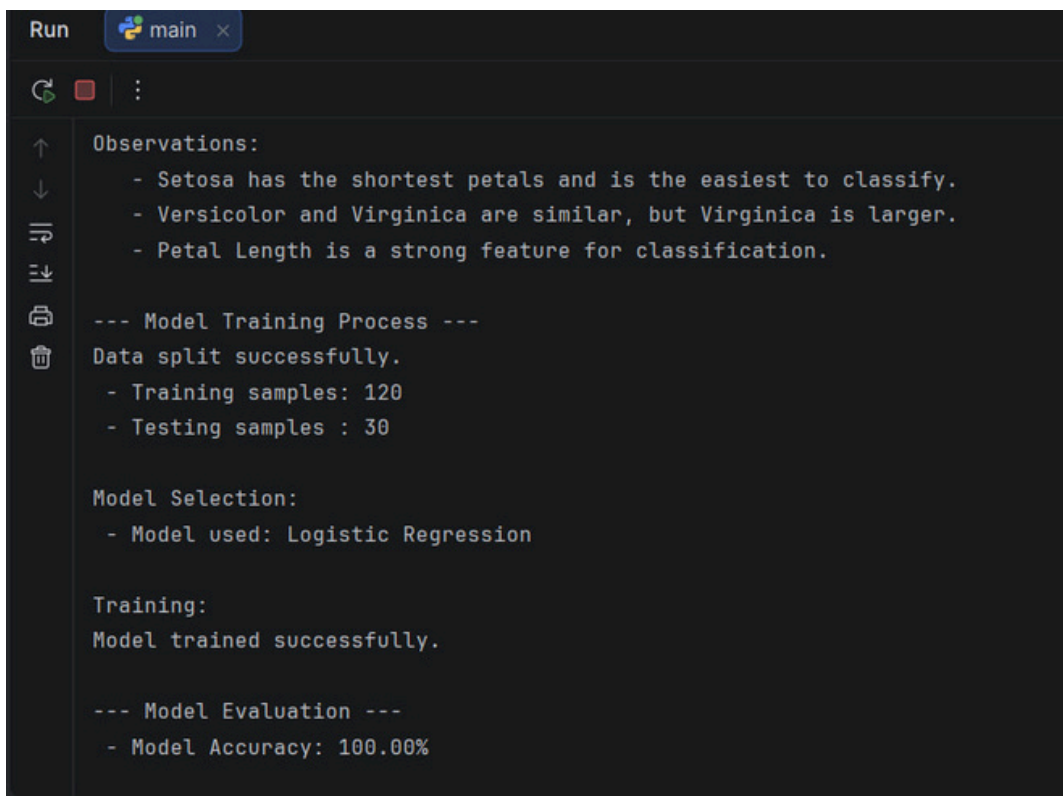
```
def evaluate_model(self): 1 usage
    if self.model is None or self.X_test is None:
        print("Error: Model not trained or data not split.")
        return
    print("\n--- Model Evaluation ---")

    y_pred = self.model.predict(self.X_test)
    accuracy = accuracy_score(self.y_test, y_pred)
    print(f" - Model Accuracy: {accuracy * 100:.2f}%")
```

The trained model is tested using unseen data, and the accuracy score is calculated to measure performance.

RESULT

The logistic regression model was evaluated on a testing set containing 30 samples. The model achieved a perfect accuracy score of 100.00%, meaning it correctly identified the species of every single sample in the test set. This high performance clearly indicates that the chosen features : sepal length, sepal width, petal length, and petal width , provide an excellent and clear separation between the three species (Setosa, Versicolor, and Virginica). No misclassifications were observed, and the model demonstrated strong generalization capability.



```
Run  main x
Observations:
- Setosa has the shortest petals and is the easiest to classify.
- Versicolor and Virginica are similar, but Virginica is larger.
- Petal Length is a strong feature for classification.

--- Model Training Process ---
Data split successfully.
- Training samples: 120
- Testing samples : 30

Model Selection:
- Model used: Logistic Regression

Training:
Model trained successfully.

--- Model Evaluation ---
- Model Accuracy: 100.00%
```

GUI IMPLEMENTATION

A graphical user interface was developed using the Tkinter library in Python. The interface includes four numeric entry fields for

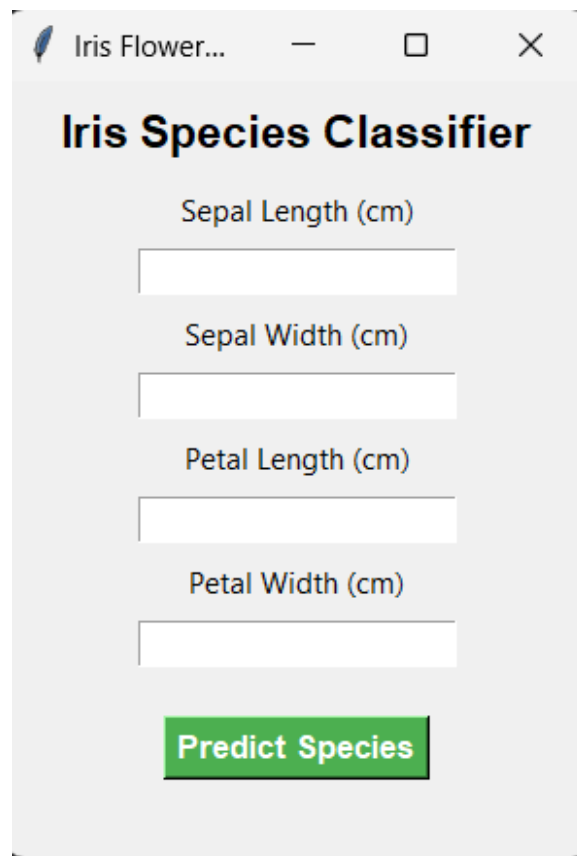
Sepal length

Sepal width

Petal length

Petal width

Users can enter plant measurements into these fields and click a button labeled "Predict Species". The system then passes these values to the trained logistic regression .model, which instantly returns the predicted species name. The result is displayed clearly on the screen, making the tool accessible for non-technical users such as students or .researchers in botany.



The screenshot shows a window titled "Iris Flower..." with standard window controls. The main heading is "Iris Species Classifier". Below it, there are four input fields, each with a label: "Sepal Length (cm)", "Sepal Width (cm)", "Petal Length (cm)", and "Petal Width (cm)". At the bottom, there is a green button with the text "Predict Species".

OOP DESIGN

The project was designed using Object-Oriented Programming (OOP) principles to ensure modularity, reusability, and ease of maintenance. The system includes the following key classes:

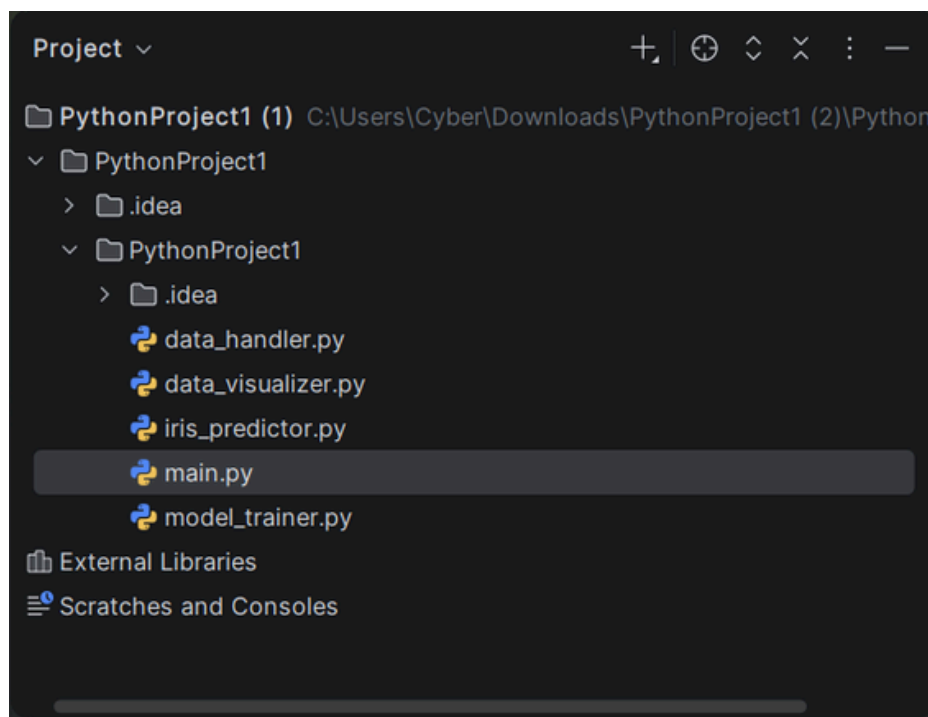
- DataHandler:** Manages dataset loading and preprocessing such as handling missing values and separating features from lab

- DataVisualizer:** Generates statistical plots including scatter plots and boxplots to support exploratory data analysis

- ModelTrainer:** Handles model selection (logistic regression) training on the training dataset, and evaluation using accuracy metrics

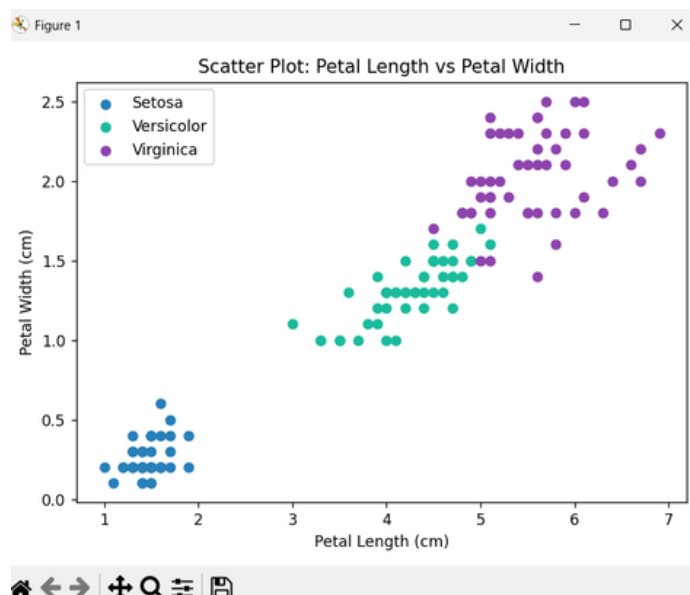
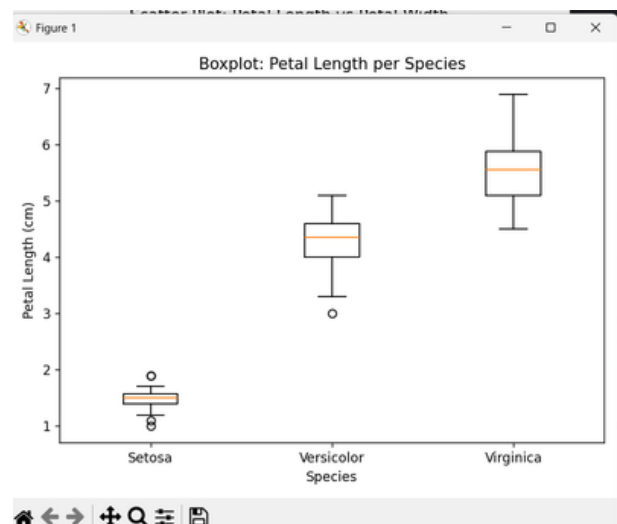
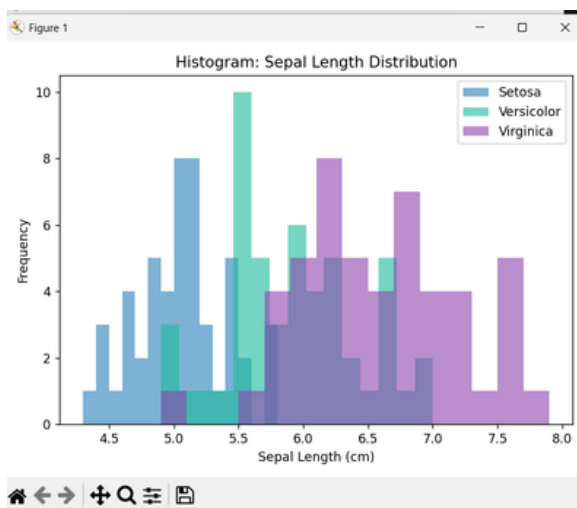
- IrisPredictor:** Connects the trained model logic to the GUI enabling real-time predictions based on user input

This OOP structure separates concerns clearly, making the code easier to debug, extend, and reuse in future projects.



DISCUSSION

The perfect accuracy of 100% is excellent, though it is not uncommon for the Iris dataset, which is known to be well-structured with clear separability between species, especially Setosa. Data visualization confirmed that Setosa is significantly smaller in petal dimensions, making it very easy to classify. Moreover, petal length and petal width proved to be the most critical features for distinguishing between the three species. If only sepal dimensions were used, accuracy would drop considerably. The integration of automated training and a user-friendly GUI makes this tool highly practical for real-world educational or light research applications



CONCLUSION

This project successfully integrated machine learning with software engineering best practices. A highly accurate (100%) logistic regression model was built, trained, and evaluated. The dataset was analyzed visually to extract meaningful insights about feature importance. The entire system was designed using OOP, resulting in modular, maintainable code. Finally, a functional desktop application with a Tkinter-based GUI allows users to input flower measurements and receive instant predictions. The use of Python's data science libraries and OOP provides a strong foundation for future IT development and project expansion.

REFERENCES

Zamindar, M. N. (n.d.). Iris flower classification using ML. Medium.

<https://medium.com/@mn.zamindar/iris-flower-classification-using-ml-29277b18b233>

Paul, A. (2025). Iris flower classification using machine learning. GitHub.

https://github.com/APaulGitHub/oibsip_taskno1

Subhajit, N. (n.d.). Iris flower classification. Kaggle.

<https://www.kaggle.com/code/nathsubhajit/iris-flower-classification>

GeeksforGeeks. (2025, July 23). Iris dataset.

<https://www.geeksforgeeks.org/machine-learning/iris-dataset/>